CONSTRUCTIVE ASYMPTOTIC ANALYSIS

Real-branch inversion in power–logarithmic scales

Explicit coefficients, rigorous remainders, convergence, and a Wolfram Language implementation

A technical study prepared for Vladimir Reshetnikov
7 September 2026

Abstract

We construct the positive-real inverse germ of

$$f(x) = ax^p \left(1 + \sum_{i=1}^{m} x^{\delta_i} P_i(\log x)\right), \qquad a, p, \delta_i > 0,$$

where the P_i are real polynomials. A normalization fixes the branch before any expansion is attempted. An explicit multi-index Lagrange–Bürmann formula produces every inverse coefficient using only polynomial differentiation. It works with irrational exponents and correctly combines coincident exponent sums. A separate truncated fixed-point algorithm computes the same coefficients by composition.

For this finite-input class the resulting grouped power–logarithmic expansion is not merely formal: it converges for all sufficiently small positive arguments. We prove this through a finite-dimensional analytic lifting, provide a usable complex contraction criterion, and derive a log-sensitive first-omitted-layer remainder. The two motivating examples, $x + x^2(1 + \log x)$ and $x + x^{\sqrt{2}}$, receive explicit formulas and detailed branch analyses. The accompanying reference package implements exact algebraic exponents, symbolic real coefficients under assumptions, powers of the inverse, and formal residual checks. Its validation record separates independently executed exact-arithmetic checks from native Wolfram tests that were supplied but could not be run in the available execution environment.

Contents

1 The problem and the two answers **3**
1.1 What is proved and what is implemented . . . 3

2 Choose the branch before choosing the series **4**
2.1 Why a complex Taylor series can answer the wrong question . . . 4
2.2 A branch theorem for the entire implemented class . . . 4

3 The finite power–logarithmic algebra **5**
3.1 Locally finite exponent supports . . . 5
3.2 Operations used by the implementation . . . 5

4 An explicit all-orders coefficient theorem **6**
4.1 Derivation by a power-core coordinate . . . 6
4.2 First and second nonlinear orders . . . 7
4.3 Polynomial degree and coefficient computation . . . 7

5 A second construction by fixed-point composition **7**

6 Why the grouped expansions actually converge **8**
6.1 A finite-dimensional analytic lifting . . . 8
6.2 An explicit sufficient convergence criterion . . . 9

7 Power cutoffs and rigorous logarithmic remainders **10**
7.1 A finite way to find the omitted layer . . . 10
7.2 Why a custom remainder object is used . . . 10

8 The logarithmic example in detail **11**
8.1 The highest logarithms are Catalan coefficients . . . 11
8.2 An explicit convergence interval . . . 11
8.3 A useful partial resummation . . . 12

9 The irrational-power example in detail **12**
9.1 Convergence radius and the complex obstruction . . . 13
9.2 Why rational approximation changes the problem . . . 13

10 Mixed sectors, nonlinear cores, and inverse observables **14**
10.1 A resonant mixed example . . . 14
10.2 A nonlinear leading power . . . 14
10.3 Powers of the inverse without repeated composition . . . 14
10.4 A cancelled omitted layer . . . 14

11 Residuals, inverse-error bounds, and forward jets **15**
11.1 The residual-to-error principle . . . 15
11.2 What the package's formal residual checks . . . 15
11.3 Using a finite forward asymptotic jet . . . 15

11.4 Newton refinement 16

12 The Wolfram Language package **16**
12.1 Loading and immediate use 16
12.2 Public functions 17
12.3 Result fields and their coordinate systems 17
12.4 Options and symbolic parameters 18
12.5 Accepted syntax and explicit failure cases 18

13 Implementation structure, invariants, and cost **19**
13.1 The construction pipeline 19
13.2 Core invariants 19
13.3 Size bounds and practical limits 19

14 Validation and reproduction **20**
14.1 Executed checks 20
14.2 Native Wolfram execution status 21
14.3 Commands to reproduce the checks and the PDF 21

15 Extensions and boundaries of the present engine **21**
15.1 Other endpoints and asymptotic coordinates 21
15.2 Leading logarithms require a different core 22
15.3 Infinite input expansions and flat corrections 22
15.4 Toward a broader engine 23

16 Relation to the supplied research materials **23**

A Compact reference algorithms **23**

B A checklist for adding a new test family **24**

C Archive contents **24**

1 The problem and the two answers

The motivating question [1] asks for the real inverse near the positive endpoint 0, rather than a Taylor expansion of an unspecified complex inverse branch. Its first function is

$$f_1(x) = x + x^2(1 + \log x), \qquad x > 0. \tag{1.1}$$

Its second example is

$$f_2(x) = x + x^{\sqrt{2}}, \qquad x > 0. \tag{1.2}$$

All logarithms of positive real arguments in this article are real natural logarithms. Unless another limit is stated, $y \to 0^+$.

Write $g_i = f_i^{-1}$ and $L = \log y$. The first answer is

$$\begin{aligned} g_1(y) = y &- y^2(1 + L) + y^3(1 + L)(3 + 2L) \\ &- \frac{y^4}{2}(1 + L)(10L^2 + 31L + 23) \\ &+ \frac{y^5}{6}(1 + L)(84L^3 + 398L^2 + 607L + 299) + \mathcal{O}\big(y^6(1 + |L|)^5\big)\,. \end{aligned} \tag{1.3}$$

In particular, after retaining only $y - y^2(1 + \log y)$, the correct ordinary big-O bound is

$$g_1(y) = y - y^2(1 + \log y) + \mathcal{O}\big(y^3(1 + |\log y|)^2\big)\,, \tag{1.4}$$

not $\mathcal{O}(y^3)$. In fact the omitted term divided by y^3 tends to $+\infty$. This is an interpretation issue, not an objection to a formal notation that explicitly declares it is grading by powers while allowing logarithmic coefficients. The package never gives an ordinary `O[y]3` object this nonstandard meaning.

Put $\alpha = \sqrt{2}$. The second answer, agreeing with the requested four-term form in the archived question, is

$$\begin{aligned} g_2(y) = y - y^\alpha + \alpha y^{2\alpha-1} &- \frac{6-\alpha}{2} y^{3\alpha-2} + \frac{17\alpha - 12}{3} y^{4\alpha-3} \\ &+ \frac{153\alpha - 305}{12} y^{5\alpha-4} + \mathcal{O}\big(y^{6\alpha-5}\big)\,. \end{aligned} \tag{1.5}$$

The increment between successive exponents is $\alpha - 1$, not an integer or a rational fraction. There is no need to approximate it by a rational number.

The rest of the article gives an all-orders construction, not just these initial terms. The implementation covers a broader finite-input class, including nonunit leading powers, multiple irrational increments, logarithmic amplitudes of arbitrary finite degree, and powers of the inverse. It is deliberately not advertised as a universal inversion engine for every elementary function or every general-support transseries.

1.1 What is proved and what is implemented

The mathematical theorems allow arbitrary fixed positive real numbers p and δ_i. The package restricts exponents, observable powers, and cutoffs to *exact real algebraic numbers*. This makes exponent ordering and collisions an effective, exact operation. Coefficients may be more general exact real constants, or real symbolic parameters with sufficient explicit assumptions.

The construction theorem, the branch theorem, and the convergence and remainder theorems below are proved for the stated class. Native Wolfram execution of the package was unavailable: its remote evaluator returned an HTTP 404 error, and no local Wolfram kernel was installed. The supplied Python reference implementation independently passed 104 exact symbolic checks; 11 high-precision numerical comparisons were also executed. The package has a separate 95-test MUnit suite, but those tests are *unrun*, not counted as passed. Detailed reproduction instructions appear in Section 14.

2 Choose the branch before choosing the series

2.1 Why a complex Taylor series can answer the wrong question

The archived question records an expansion with constant term

$$c = \exp(-1 + W_{-1}(-\mathrm{e})).$$

This is not a small positive real number. The equation for a nonzero zero of f_1 is $1 + x(1 + \log x) = 0$. Setting $u = 1 + \log x$ gives $u\mathrm{e}^u = -\mathrm{e}$, so suitable complex Lambert branches produce such zeros. Expanding a local inverse around one of those regular complex zeros can be perfectly coherent while having nothing to do with the inverse germ that satisfies $g(y) \to 0^+$.

A condition on the target such as $y > 0$ does not, by itself, specify which source germ is being inverted. The useful branch data are

$$x > 0, \quad y > 0, \quad x \to 0, \quad \frac{x}{(y/a)^{1/p}} \to 1. \tag{2.1}$$

The last condition distinguishes the desired normalized germ. No complex-root enumeration is necessary.

The behavior reported in the question is a historical observation about the displayed commands. It is not evidence that every current Wolfram version must behave the same way. The official documentation describes `InverseSeries` and `AsymptoticSolve` as general tools in their respective domains [6, 7]. Our purpose is to give an explicit algorithm with a precise supported class and branch contract, not to benchmark unexecuted current built-ins.

2.2 A branch theorem for the entire implemented class

Theorem 2.1 (The positive inverse germ)**.** *Suppose*

$$f(x) = ax^p(1 + H(x)), \qquad H(x) = \sum_{i=1}^{m} x^{\delta_i} P_i(\log x), \tag{2.2}$$

where $a, p, \delta_i > 0$ and each $P_i \in \mathbb{R}[\ell]$. There is an $\varepsilon > 0$ such that f is positive and strictly increasing on $(0, \varepsilon)$. It has a unique inverse on its image, and this inverse satisfies Equation (2.1).

Proof. Every $x^\delta P(\log x)$ with $\delta > 0$ tends to zero. Moreover,

$$xH'(x) = \sum_i x^{\delta_i} \left(\delta_i P_i(\log x) + P_i'(\log x)\right) = o(1). \tag{2.3}$$

Thus $1 + H(x) > 0$ eventually and

$$f'(x) = ax^{p-1}\{p(1 + H(x)) + xH'(x)\} = apx^{p-1}(1 + o(1)) > 0.$$

Continuity, the limit $f(x) \to 0$, and strict monotonicity give the local inverse. From $y = ag(y)^p(1+o(1))$ and positivity, taking the real pth root gives $g(y)/(y/a)^{1/p} \to 1$. □

For Equation (1.1) the conclusion is global. Indeed,

$$f_1'(x) = 1 + x(3 + 2\log x) \geq 1 - 2\mathrm{e}^{-5/2} > 0, \qquad x > 0. \tag{2.4}$$

The minimum follows by differentiating $x(3 + 2\log x)$. Since $f_1(0^+) = 0$ and $f_1(x) \to \infty$ as $x \to \infty$, it is a bijection of $(0, \infty)$ onto itself. Likewise $f_2'(x) = 1 + \sqrt{2}\, x^{\sqrt{2}-1} > 1$, so its positive inverse is globally unique.

Although $g_1(0) = 0$ gives a C^1 extension with $g_1'(0) = 1$, that extension is not C^2:

$$g_1''(y) = -\frac{5 + 2\log g_1(y)}{f_1'(g_1(y))^3} \longrightarrow +\infty.$$

An ordinary Taylor series at the endpoint is therefore the wrong representation even after the branch has been fixed.

3 The finite power–logarithmic algebra

3.1 Locally finite exponent supports

Let

$$\Sigma = \left\{ \sum_{i=1}^{m} k_i \delta_i : \boldsymbol{k} = (k_1, \dots, k_m) \in \mathbb{N}^m \right\}, \qquad \delta_* = \min_i \delta_i, \tag{3.1}$$

where $0 \in \mathbb{N}$. For every finite B, the set $\Sigma \cap [0, B]$ is finite: $k_i \delta_i \le B$ bounds each k_i. This is true even if the additive *group* generated by the δ_i is dense in $\mathbb{R}$. Nonnegative coefficients, rather than arbitrary integer coefficients, are essential.

Several distinct multi-indices can give the same weight. For example, with increments $1/2$ and 1, the weight 1 is produced both by $(2, 0)$ and by $(0, 1)$. Their coefficient polynomials must be added before a term or a remainder is reported. Algebraic equivalences such as $\sqrt{8}/2 = \sqrt{2}$ must also be recognized exactly.

A normalized formal expression has the form

$$U(t) = \sum_{\Delta \in \Sigma} t^{\Delta} Q_{\Delta}(\ell), \qquad \ell = \log t, \quad Q_{\Delta} \in \mathbb{R}[\ell]. \tag{3.2}$$

At each bounded weight there are only finitely many products to compute. The coefficient ring is a polynomial ring in ℓ; it is not a ring of coefficients constant with respect to differentiation in t.

For genuine asymptotic comparison, a smaller power of t dominates every fixed logarithmic degree at a larger power. Within one power, the highest nonzero logarithmic degree dominates. For a nonzero polynomial Q of degree d,

$$t^{\alpha+\eta} Q(\log t) = o(t^{\alpha}), \qquad \eta > 0.$$

This observation explains both the finite cutoff algorithm and the need to preserve logarithmic degrees in ordinary remainder bounds.

3.2 Operations used by the implementation

Addition merges equal powers and adds their polynomials. Multiplication is finite convolution at every cutoff:

$$[t^{\Delta}](UV) = \sum_{\beta+\gamma=\Delta} Q_{\beta}(\ell) R_{\gamma}(\ell).$$

For a positive-order jet U, generalized binomial and logarithmic expansions are well-defined by

$$(1+U)^r = \sum_{n \ge 0} \binom{r}{n} U^n, \tag{3.3}$$

$$\log(1+U) = \sum_{n \ge 1} \frac{(-1)^{n+1}}{n} U^n. \tag{3.4}$$

If $\operatorname{val} U \ge \delta > 0$, only $n \le B/\delta$ can contribute to weights at most B. This makes those infinite formulas finite algorithms.

The differentiation rule is

$$\frac{\mathrm{d}}{\mathrm{d}t}\{t^{\beta} Q(\ell)\} = t^{\beta-1}(\beta + \partial_{\ell}) Q(\ell). \tag{3.5}$$

Iterating it gives

$$\frac{\mathrm{d}^r}{\mathrm{d}t^r}\{t^{\beta} Q(\ell)\} = t^{\beta-r} \prod_{j=0}^{r-1} (\beta - j + \partial_{\ell}) Q(\ell). \tag{3.6}$$

The factors commute because they are constant-coefficient operators in ℓ. Differentiation does not increase logarithmic degree.

Finally, composition into tV with $V = 1 + U$ uses identities justified on the positive branch:

$$(tV)^\delta = t^\delta V^\delta, \qquad \log(tV) = \ell + \log V. \tag{3.7}$$

There is no unrestricted complex `PowerExpand` step. Formally, Equations (3.3) and (3.4) define the right-hand side; analytically, V is positive and close to 1.

4 An explicit all-orders coefficient theorem

The classical Lagrange–Bürmann formula [2] is used here with an *auxiliary perturbation parameter*. It is not applied as an ordinary Taylor theorem at a singular logarithmic endpoint. This is the same phase-coordinate strategy used in the supplied companion notes [3], specialized to a power core and polynomial logarithmic amplitudes.

Let

$$t = (y/a)^{1/p}, \qquad \ell = \log t = \frac{\log(y/a)}{p}. \tag{4.1}$$

For $\boldsymbol{k} \in \mathbb{N}^m$ set

$$n = |\boldsymbol{k}|, \quad \Delta = \boldsymbol{k} \cdot \boldsymbol{\delta}, \quad \boldsymbol{k}! = \prod_i k_i!, \quad P_{\boldsymbol{k}}(\ell) = \prod_i P_i(\ell)^{k_i}. \tag{4.2}$$

Theorem 4.1 (Power–logarithmic inverse coefficients)**.** *For the positive inverse germ g of Equation (2.2), and any fixed $q \in \mathbb{R}$, one has the formal expansion*

$$\boxed{g(y)^q = t^q + \sum_{\boldsymbol{k} \neq \boldsymbol{0}} t^{q+\Delta} \frac{(-1)^n q}{p^n \boldsymbol{k}!} \prod_{j=1}^{n-1} (q + \Delta + pj + \partial_\ell) P_{\boldsymbol{k}}(\ell).} \tag{4.3}$$

The product is the identity for $n = 1$. Equal values of Δ are combined. At every finite power cutoff this is a finite computation. For the exact finite model Equation (2.2), the expansion converges to $g(y)^q$ for all sufficiently small $y > 0$.

The proof of the coefficient identity follows immediately below. Convergence, including the justification for using physical marker values rather than merely small formal markers, is proved independently in Section 6. The special case $q = 0$ gives the exact constant 1; its apparent correction terms vanish through the factor q.

4.1 Derivation by a power-core coordinate

Put $s = x^p$ and $Y = y/a$. With independent markers ε_i define

$$h(s) = \sum_i \varepsilon_i \, s^{1+\delta_i/p} P_i\left(\frac{\log s}{p}\right).$$

The equation becomes $Y = s + h(s)$. For fixed $Y > 0$, all functions involved are analytic on a sufficiently small complex disk about $s = Y$, using the logarithm continued from the positive axis. For the local solution near $s = Y$, Lagrange–Bürmann applied to the observable $s^{q/p}$ gives

$$s^{q/p} = Y^{q/p} + \sum_{n \geq 1} \frac{(-1)^n}{n!} \frac{\mathrm{d}^{n-1}}{\mathrm{d}Y^{n-1}} \left\{ \frac{q}{p} Y^{q/p-1} h(Y)^n \right\}. \tag{4.4}$$

This is an identity in small markers and also a formal marker identity. The multinomial coefficient for $\boldsymbol{\varepsilon}^{\boldsymbol{k}}$ in h^n is $n!/\boldsymbol{k}!$. Before the $n-1$ derivatives, its power of Y is

$$\frac{q+\Delta}{p}+n-1.$$

Each derivative lowers this power by one and acts on the logarithmic polynomial by $p^{-1}\partial_\ell$. Applying Equation (3.6) leaves $Y^{(q+\Delta)/p}=t^{q+\Delta}$ and the operator factors

$$\prod_{j=1}^{n-1}\left(\frac{q+\Delta}{p}+j+\frac{1}{p}\partial_\ell\right).$$

Together with the initial q/p and the multinomial factor, this is exactly Equation (4.3). Local finiteness permits collection by the intended power weights. The convergence argument in Section 6 then allows every marker to be set to 1 for sufficiently small t.

4.2 First and second nonlinear orders

The first inverse correction is

$$g(y)=t-\frac{1}{p}\sum_i t^{1+\delta_i}P_i(\ell)+\cdots. \tag{4.5}$$

At total marker degree two, a repeated sector i contributes

$$\frac{t^{1+2\delta_i}}{2p^2}(1+2\delta_i+p+\partial_\ell)P_i(\ell)^2,$$

whereas distinct sectors $i<j$ contribute

$$\frac{t^{1+\delta_i+\delta_j}}{p^2}(1+\delta_i+\delta_j+p+\partial_\ell)P_i(\ell)P_j(\ell).$$

These are orders in the perturbation markers, not necessarily adjacent orders in asymptotic dominance. A single large increment can occur after many repetitions of a smaller increment. The algorithm therefore sorts by the actual weight, not by total degree.

4.3 Polynomial degree and coefficient computation

If $d_i=\deg P_i$, the polynomial contributed by $\boldsymbol{k}$ has degree at most $\sum_i k_i d_i$. For an individual operator $c+\partial_\ell$, a coefficient vector $(b_0,\dots,b_d)$ is updated by

$$\widetilde{b}_r=cb_r+(r+1)b_{r+1},\qquad b_{d+1}=0. \tag{4.6}$$

Thus the potentially intimidating derivative in Lagrange inversion is only a sequence of finite polynomial transformations. The package uses Wolfram's polynomial `D` and `Expand` operations; a future low-level coefficient-vector backend can implement Equation (4.6) without changing the theorem or public data model.

5 A second construction by fixed-point composition

Set $g(y)=tV(t)$. The normalized equation is

$$V=\left(1+\sum_i t^{\delta_i}V^{\delta_i}P_i(\ell+\log V)\right)^{-1/p}. \tag{5.1}$$

Starting from $V_0=1$, define V_{r+1} by the right-hand side, truncating all jet operations to the requested finite weight bound.

Proposition 5.1 (Finite stabilization). *If two normalized jets V and W agree below relative weight $\beta > 0$, their images under Equation (5.1) agree below weight $\beta + \delta_*$. Consequently V_r agrees with the formal solution at all weights strictly below $(r+1)\delta_*$. In particular, $\lfloor B/\delta_* \rfloor + 1$ iterations suffice for all weights at most B.*

Proof. For $V, W \in 1 + \mathfrak{m}$, the difference between V^δ and W^δ is $(V - W)$ times a formal series of nonnegative order. The same is true of $\log V - \log W$, by the binomial and logarithmic identities. Substituting these into a polynomial P preserves that property. Each correction is then multiplied by t^{δ_i}, so its difference gains at least δ_*. The outer power $-1/p$ has a nonzero constant base and does not reduce order. Iteration gives the assertion, beginning with $V - 1$ of order at least δ_*. □

This also proves formal uniqueness. If two solutions first differed at weight β, applying the equation would force their difference to start no earlier than $\beta + \delta_*$, a contradiction. Thus the fixed-point construction and Equation (4.3) compute the same formal object.

For the near-identity case $p = a = 1$, an alternative unnormalized iteration is $G_{r+1}(y) = y - h(G_r(y))$, with $h = f - x$. The normalized iteration is more uniform: it also handles $p \neq 1$ and avoids taking a noninteger power of a series with zero constant term. Its internal powers are always powers of $1+$ a positive-order jet.

The two algorithms have different failure modes in an implementation. A missing derivative in Equation (4.3) can be detected by composition; a missing cross term in composition can be detected by the explicit multi-index formula. Their agreement is therefore a useful test, although it does not replace the proofs.

6 Why the grouped expansions actually converge

6.1 A finite-dimensional analytic lifting

Write $P_i(\ell) = \sum_{k=0}^{d_i} c_{ik} \ell^k$. Expanding $P_i(\ell + \log V)$ gives

$$t^{\delta_i} P_i(\ell + \log V) = \sum_{j=0}^{d_i} z_{ij} A_{ij}(\log V), \quad z_{ij} = t^{\delta_i} \ell^j, \tag{6.1}$$

where

$$A_{ij}(u) = \sum_{k=j}^{d_i} c_{ik} \binom{k}{j} u^{k-j}.$$

There are finitely many independent variables z_{ij}. Consider

$$F(V, \boldsymbol{z}) = V^p \left(1 + \sum_{i,j} z_{ij} V^{\delta_i} A_{ij}(\log V) \right) - 1. \tag{6.2}$$

Choose the complex logarithm analytic on a disk centered at 1 that does not meet 0. Then F is analytic near $(V, \boldsymbol{z}) = (1, \mathbf{0})$, $F(1, \mathbf{0}) = 0$, and $F_V(1, \mathbf{0}) = p \neq 0$.

Theorem 6.1 (Convergent analytic lifting). *There is an analytic function $V = \mathcal{V}(\boldsymbol{z})$ near $\boldsymbol{z} = 0$ with $\mathcal{V}(0) = 1$ satisfying Equation (6.2). Its Taylor series converges absolutely in a sufficiently small polydisk. Along the real curve $z_{ij} = t^{\delta_i} (\log t)^j$, all variables tend to zero, so $t\mathcal{V}(\boldsymbol{z}(t))$ is the positive inverse for sufficiently small $t > 0$. Grouping the Taylor series by exponent weights gives Equation (4.3), and the grouped expansion converges absolutely after its polynomial amplitudes are evaluated at $\ell = \log t$.*

Proof. The analytic implicit function theorem gives $\mathcal{V}$. Each $z_{ij}(t)$ tends to zero because $\delta_i > 0$. For real sufficiently small t, the solution remains real and close to 1, hence positive; Theorem 2.1 identifies it with the required inverse. Every monomial in the Taylor series has weight $\sum_i k_i\delta_i$ after substitution, with a finite logarithmic polynomial obtained by collecting the choices of j. Absolute convergence allows regrouping. Formal uniqueness from Theorem 5.1, or the marker derivation above, identifies the grouped coefficients with Equation (4.3). □

It is useful to distinguish this theorem from a general assertion about transseries. General transseries need not converge as ordinary sums, and formal identities do not automatically justify evaluation of arbitrary marker values. Our conclusion uses the *finite* number of power–logarithmic inputs and the analytic function near $V = 1$. The broader grid-based viewpoint is discussed in [4, 5]; no unproved general-support closure statement is needed here.

6.2 An explicit sufficient convergence criterion

The preceding proof is qualitative. A contraction estimate gives a computable sufficient condition at a specified positive t. Fix $0 < r < 1$ and define

$$M_r = -\log(1-r), \qquad P_i^{\#}(u) = \sum_k |c_{ik}|u^k.$$

For $\ell = \log t$ set

$$A(t) = \sum_i t^{\delta_i}(1+r)^{\delta_i} P_i^{\#}(|\ell| + M_r), \tag{6.3}$$

$$B(t) = \frac{1}{1-r}\sum_i t^{\delta_i}(1+r)^{\delta_i}\left\{\delta_i P_i^{\#}(|\ell|+M_r) + (P_i^{\#})'(|\ell|+M_r)\right\}. \tag{6.4}$$

On $|V-1| \le r$, these bound respectively $|H(tV)|$ and its derivative with respect to V. The estimates follow from $|\log V| \le M_r$, $|V| \le 1+r$, and $|V| \ge 1-r$.

Proposition 6.2 (Marker disk and tail bound)**.** *Let $R > 1$. If $RA(t) < 1$ and*

$$\frac{RA(t)}{p}(1-RA(t))^{-1/p-1} \le r, \tag{6.5}$$

$$\kappa := \frac{RB(t)}{p}(1-RA(t))^{-1/p-1} < 1, \tag{6.6}$$

then the normalized inverse is analytic in the common perturbation marker ε for $|\varepsilon| \le R$ and remains in $|V-1| \le r$. If all multi-index terms of total degree at most N are retained, then

$$|g(y) - G_N(y)| \le t\,\frac{rR^{-(N+1)}}{1-R^{-1}}. \tag{6.7}$$

This bounds a total-marker-degree truncation, not an arbitrary weighted cutoff without further accounting.

Proof. The map $T(V) = (1+\varepsilon H(tV))^{-1/p}$ is analytic on the disk because $|\varepsilon H| \le RA < 1$. Integrating its derivative with respect to εH along the straight segment from 0 shows that $|T(V)-1|$ is bounded by the left side of Equation (6.5). Its V-derivative is bounded by Equation (6.6). The map is a uniform contraction of the closed disk into itself. Iteration gives its unique fixed point, analytically in ε. Cauchy's estimate on $V-1$, whose absolute value is at most r, bounds its nth marker coefficient by rR^{-n}. Summing the geometric tail and multiplying by t proves Equation (6.7). □

All quantities in Equations (6.3) and (6.4) tend to zero as $t \to 0^+$, so the criterion succeeds eventually for any fixed r and R. It is a sufficient test; failure does not imply divergence.

7 Power cutoffs and rigorous logarithmic remainders

The public cutoff b means: retain every term whose *target-variable power* is strictly less than b. For an observable g^q, a normalized term $t^{q+\Delta}Q_\Delta(\ell)$ has target power $(q+\Delta)/p$. Thus the relative weight threshold is

$$B = pb - q > 0. \tag{7.1}$$

Define the first possible omitted weight

$$\lambda = \min\{\Delta \in \Sigma : \Delta \geq B\}. \tag{7.2}$$

The associated coefficient is the *combined* polynomial Q_λ, which can vanish through cancellation.

Theorem 7.1 (First omitted layer). *Let $G_{<b}$ contain every term of $g(y)^q$ of target power less than b. For the exact finite model,*

$$g(y)^q - G_{<b}(y) = t^{q+\lambda}Q_\lambda(\ell) + o(t^{q+\lambda}). \tag{7.3}$$

If $Q_\lambda \neq 0$ and $d = \deg Q_\lambda$, then

$$g(y)^q - G_{<b}(y) = \mathcal{O}\Big(t^{q+\lambda}(1+|\log t|)^d\Big). \tag{7.4}$$

If $Q_\lambda = 0$, the bound $\mathcal{O}(t^{q+\lambda})$ is valid, though generally not sharp.

Proof. Local finiteness gives a positive gap between λ and the next possible weight. After removing finitely many layers, the remaining Taylor series of Equation (6.2) has arbitrarily high total degree in the finite variables z_{ij}. If $d_* = \max_i d_i$, their largest absolute value is bounded by a constant times $t^{\delta_*}(1+|\log t|)^{d_*}$. Choose the removed Taylor degree N so large that $(N+1)\delta_* > \lambda$. The analytic Taylor tail is then $o(t^\lambda)$. The finitely many remaining layers above λ are also $o(t^\lambda)$ by their positive power gaps. Multiplication by t^q proves Equation (7.3). The polynomial degree gives the stated ordinary big-O bounds. □

7.1 A finite way to find the omitted layer

Let $n_* = \lceil B/\delta_* \rceil$. The semigroup contains $n_*\delta_* \geq B$, so

$$\lambda \leq n_*\delta_*.$$

It therefore suffices to enumerate multi-indices satisfying

$$\boldsymbol{k}\cdot\boldsymbol{\delta} \leq n_*\delta_*. \tag{7.5}$$

This gives all retained weights and the full first omitted bucket. It also gives $|\boldsymbol{k}| \leq n_*$. Importantly, a numerical approximation to δ_* is neither needed nor safe at a boundary.

The package returns both the polynomial Q_λ and its degree. If that polynomial cancels, it does not silently claim to have found the first *nonzero* omitted layer. A zero frontier still supplies the conservative bound in Theorem 7.1. Searching farther would improve sharpness, not the correctness of the retained expansion.

7.2 Why a custom remainder object is used

`PowerLogRemainder[t,r,d]` means only $\mathcal{O}(t^r(1+|\log t|)^d)$ on the positive axis. It is an inert descriptor, not a `SeriesData` object and not an implemented algebra of error terms. The returned association separately stores the finite expression, its terms, the target power of the remainder, and its logarithmic degree. Adding two such descriptors does not magically combine their bounds. This deliberately small interface prevents an attractive display from making an unsupported mathematical promise.

8 The logarithmic example in detail

For f_1, the model data are $a = p = 1$, one increment $\delta = 1$, and $P(\ell) = 1 + \ell$. The all-orders formula reduces to

$$g_1(y) = y + \sum_{n\geq 1} y^{n+1} A_n(L), \qquad A_n(L) = \frac{(-1)^n}{n!} \prod_{j=1}^{n-1} (n + 1 + j + \partial_L)(1 + L)^n. \tag{8.1}$$

The first five polynomials are

$$\begin{aligned}
A_1 &= -(L+1),\\
A_2 &= (L+1)(2L+3),\\
A_3 &= -\tfrac{1}{2}(L+1)(10L^2+31L+23),\\
A_4 &= \tfrac{1}{6}(L+1)(84L^3+398L^2+607L+299),\\
A_5 &= -\tfrac{1}{12}(L+1)(504L^4+3223L^3+7507L^2+7565L+2789).
\end{aligned}$$

Every polynomial A_n has the factor $L + 1$: at most $n - 1$ differentiations are applied to $(L + 1)^n$. This is consistent with the exact fixed point $f_1(\mathrm{e}^{-1}) = \mathrm{e}^{-1}$, although the asymptotic construction concerns the endpoint 0 rather than that fixed point.

8.1 The highest logarithms are Catalan coefficients

Only the constant part of each operator contributes to the coefficient of L^n. Consequently

$$[L^n]A_n(L) = \frac{(-1)^n}{n!} \prod_{j=1}^{n-1} (n + 1 + j) = (-1)^n \frac{1}{n+1} \binom{2n}{n}. \tag{8.2}$$

Thus the leading-logarithm diagonal is controlled by Catalan numbers. This is not an empirical pattern inferred from a few coefficients; it follows for every n from the differential-operator formula.

Keeping through A_N gives

$$g_1(y) = y + \sum_{n=1}^{N} y^{n+1} A_n(\log y) + \mathcal{O}\big(y^{N+2}(1 + |\log y|)^{N+1}\big)\,. \tag{8.3}$$

The degree $N + 1$ is sharp for this ordinary power–log envelope, because its highest coefficient Equation (8.2) is nonzero.

8.2 An explicit convergence interval

Apply Theorem 6.2 with $r = 1/2$ and $R = 2$. Here $t = y$ and

$$A(t) = \tfrac{3}{2}t(1 + |\log t| + \log 2), \qquad B(t) = 3t(2 + |\log t| + \log 2).$$

Both displayed functions increase on $(0, 1/100]$. Using $\log 100 < 5$ and $\log 2 < 1$ gives throughout that interval

$$A \leq \frac{21}{200}, \qquad B \leq \frac{6}{25}.$$

The disk displacement bound and contraction constant therefore satisfy

$$\frac{2A}{(1-2A)^2} \leq \frac{2100}{6241} < \frac{1}{2}, \qquad \frac{2B}{(1-2A)^2} \leq \frac{4800}{6241} < 1.$$

This proves convergence of Equation (8.1) for every $0 < y \le 1/100$, with the explicit, deliberately conservative bound

$$\left| g_1(y) - y - \sum_{n=1}^{N} y^{n+1} A_n(\log y) \right| \le y\, 2^{-(N+1)}. \tag{8.4}$$

The power–log bound Equation (8.3) is much more informative as $y \to 0$ at fixed N; Equation (8.4) instead guarantees convergence as $N \to \infty$ on an explicit interval. These are different uses of a remainder estimate.

8.3 A useful partial resummation

The analytic lifting for this example has only two variables,

$$z = y \log y, \qquad w = y.$$

Its equation is

$$V + (z + w)V^2 + wV^2 \log V = 1. \tag{8.5}$$

At $w = 0$, the branch near $V = 1$ is

$$V_0(z) = \frac{2}{1 + \sqrt{1 + 4z}}. \tag{8.6}$$

The rationalized form avoids subtracting nearly equal numbers at $z = 0$. Differentiating Equation (8.5) with respect to w at $w = 0$ gives

$$\left.\frac{\partial V}{\partial w}\right|_{w=0} = -\frac{V_0(z)^2(1 + \log V_0(z))}{1 + 2zV_0(z)} = -\frac{V_0(z)^2(1 + \log V_0(z))}{\sqrt{1 + 4z}}.$$

Hence, as $y \to 0^+$,

$$g_1(y) = yV_0(y \log y) - y^2 \frac{V_0(y \log y)^2\{1 + \log V_0(y \log y)\}}{\sqrt{1 + 4y \log y}} + \mathcal{O}(y^3). \tag{8.7}$$

Here an ordinary $\mathcal{O}(y^3)$ really is valid: infinitely many leading and subleading logarithms have been retained in exact analytic coefficient functions. The analytic Taylor remainder in w is uniform for z in a small fixed disk, and $z = y \log y$ eventually lies there. This resummation is a theoretical and numerical alternative, not an extra package backend. The package returns finite power–logarithmic truncations.

9 The irrational-power example in detail

Consider the more general family

$$f_\alpha(x) = x + cx^\alpha, \qquad \alpha > 1, \tag{9.1}$$

with fixed real c; positivity and monotonicity hold near zero even when $c < 0$. Put $\delta = \alpha - 1$. The coefficient theorem gives

$$g_\alpha(y) = y \sum_{n \ge 0} C_n(\alpha)(cy^\delta)^n, \tag{9.2}$$

where $C_0 = 1$ and

$$C_n(\alpha) = \frac{(-1)^n}{n!} \prod_{j=0}^{n-2} (n\alpha - j) \tag{9.3}$$

$$= \frac{(-1)^n}{1 + n(\alpha - 1)} \binom{n\alpha}{n} \tag{9.4}$$

$$= (-1)^n \frac{\Gamma(n\alpha + 1)}{\Gamma(n + 1)\Gamma(n(\alpha - 1) + 2)}. \tag{9.5}$$

The finite product is the preferred exact computational formula. It avoids unnecessary gamma functions and their associated simplification and pole-cancellation issues.

The first coefficients are

$$\begin{aligned} C_1 &= -1, \qquad C_2 = \alpha, \\ C_3 &= -\frac{\alpha(3\alpha-1)}{2}, \\ C_4 &= \frac{\alpha(4\alpha-1)(2\alpha-1)}{3}. \end{aligned}$$

Substituting $\alpha = \sqrt{2}$, $c = 1$ gives Equation (1.5). An ordinary power cutoff $b > 1$ retains exactly those integers $n \geq 0$ with $1 + n\delta < b$. This is a strict inequality; using a floating-point `Floor` at an exact boundary can retain or omit the wrong term.

9.1 Convergence radius and the complex obstruction

Write $z = cy^\delta$ and $g = yV(z)$. Then

$$V + zV^\alpha = 1, \qquad V(0) = 1. \tag{9.6}$$

This defines an ordinary analytic function of z near zero, even when α is irrational. Stirling's formula applied to Equation (9.5) yields

$$|C_n(\alpha)| \sim \frac{\sqrt{\alpha}}{\sqrt{2\pi}(\alpha-1)^{3/2}} n^{-3/2} \left(\frac{\alpha^\alpha}{(\alpha-1)^{\alpha-1}}\right)^n. \tag{9.7}$$

Therefore the exact radius of convergence in z is

$$R_\alpha = \frac{(\alpha-1)^{\alpha-1}}{\alpha^\alpha}. \tag{9.8}$$

The same value is suggested by the critical point of the implicit equation: simultaneously solving $V + zV^\alpha = 1$ and $1 + \alpha z V^{\alpha-1} = 0$ gives

$$V_c = \frac{\alpha}{\alpha-1}, \qquad z_c = -R_\alpha.$$

The coefficient asymptotic, rather than an unproved assertion that this is the nearest singularity, establishes the radius. Because the boundary coefficients have size proportional to $n^{-3/2}$, the series converges absolutely on $|z| = R_\alpha$ as well.

For $\alpha = \sqrt{2}$ the numerical values are

$$R_\alpha \approx 0.42519560665184081380, \qquad R_\alpha^{1/(\alpha-1)} \approx 0.12686270512330968140. \tag{9.9}$$

For $c = 1$, the positive inverse exists for every $y > 0$, but this particular series centered at $z = 0$ has a finite convergence radius. Global invertibility and convergence of one local expansion are separate facts.

9.2 Why rational approximation changes the problem

Replacing $\sqrt{2}$ by a nearby rational number makes a Puiseux grid available, but it computes the inverse of a different function. If two exponents differ by a fixed nonzero η, the ratio of their monomials is y^η, which does not tend to 1 as $y \to 0$. At sufficiently small y, even a very small fixed exponent error changes asymptotic dominance. The package consequently rejects inexact exponents instead of silently rationalizing them.

10 Mixed sectors, nonlinear cores, and inverse observables

10.1 A resonant mixed example

Let

$$f(x) = x + x^{3/2} + 2x^2 \log x.$$

The increments are $1/2$ and 1. At relative weight 1, the quadratic interaction of the first sector contributes $3/2$, while the second sector contributes $-2\log y$. At relative weight $3/2$, a cubic term and a mixed term must also be combined. The result is

$$\begin{aligned} g(y) = y - y^{3/2} + y^2\left(\frac{3}{2} - 2\log y\right) + y^{5/2}\left(7\log y - \frac{5}{8}\right) \\ + \mathcal{O}\big(y^3(1+|\log y|)^2\big)\,. \end{aligned} \tag{10.1}$$

Enumerating one term for each input sector would miss both interactions. Treating equal weights as distinct ordered monomials would also give the wrong remainder semantics.

10.2 A nonlinear leading power

Take

$$f(x) = 2x^3\left(1 + x^{1/2}(1+\log x)\right), \qquad t = (y/2)^{1/3}, \quad \ell = \log t.$$

The positive inverse is

$$g(y) = t - \frac{1+\ell}{3}t^{3/2} + \frac{(1+\ell)(5\ell+7)}{18}t^2 + \mathcal{O}\Big(t^{5/2}(1+|\ell|)^3\Big)\,. \tag{10.2}$$

The target powers are $1/3$, $1/2$, and $2/3$; the next is $5/6$. The logarithmic coefficient is evaluated at $\log(y/2)/3$, not at $\log y$. Omitting the constant a or the factor p in the logarithm changes coefficients, even though the leading order may look plausible.

10.3 Powers of the inverse without repeated composition

Setting $q = 2$ in Equation (4.3) directly computes g^2; it does not first construct a longer expansion of g and square it. For instance, for $f(x) = x + x^2$,

$$g(y)^2 = y^2 - 2y^3 + 5y^4 - 14y^5 + \mathcal{O}(y^6).$$

Negative and fractional q are equally valid on the positive branch. The only change to cutoff bookkeeping is $B = pb - q$. This makes the observable formula useful for quantities that depend on the inverse without requiring the inverse as an intermediate output.

10.4 A cancelled omitted layer

For $f(x) = x + x^2 + 2x^3$, the inverse starts

$$g(y) = y - y^2 + 0\,y^3 + 5y^4 + \cdots\,.$$

A strict cutoff $b = 3$ retains $y - y^2$. The first possible omitted weight is 2, but its coefficient is zero. Reporting $\mathcal{O}(y^3)$ is correct and conservative; claiming that the error is asymptotic to a nonzero multiple of y^3 would be false. The result association explicitly exposes this cancellation in `"FrontierPolynomial"`.

11 Residuals, inverse-error bounds, and forward jets

11.1 The residual-to-error principle

Proposition 11.1 (A posteriori inverse bound)**.** *Let f be continuously differentiable and increasing on an interval I, with $f'(x) \geq m > 0$ there. Suppose the true inverse value $g(y)$ and an approximation $G(y)$ both belong to I. Then*

$$|G(y) - g(y)| \leq \frac{|f(G(y)) - y|}{m}. \tag{11.1}$$

Proof. The mean value theorem gives $f(G) - f(g) = f'(\xi)(G - g)$ for a point between G and g. Taking absolute values proves the assertion. □

For a general power core, the relevant slope is not a fixed constant: it is asymptotic to apt^{p-1}. On a comparability interval such as $[t/2, 2t]$, sufficiently small t gives a lower bound ct^{p-1} with $c > 0$. Thus

$$f(G) - y = \mathcal{O}(t^{p+\beta}(1 + |\log t|)^d) \quad \Longrightarrow \quad G - g = \mathcal{O}(t^{1+\beta}(1 + |\log t|)^d). \tag{11.2}$$

Forgetting the factor t^{p-1} produces a wrong inverse remainder when $p \neq 1$.

For the two motivating examples, there are particularly simple global bounds for every positive approximation G:

$$|G - g_1(y)| \leq \frac{|f_1(G) - y|}{1 - 2\mathrm{e}^{-5/2}}, \tag{11.3}$$

$$|G - g_2(y)| \leq |f_2(G) - y|. \tag{11.4}$$

These are exact mathematical inequalities. Turning a floating-point residual evaluation into a certified numerical error bar additionally requires enclosing its rounding error, for example with validated interval arithmetic. The included high-precision numerical experiments are not claimed to be such interval certificates.

11.2 What the package's formal residual checks

For $G = tV$, the normalized residual is

$$\mathcal{R}(t) = \frac{f(G)}{at^p} - 1 = V^p \left(1 + \sum_i t^{\delta_i} V^{\delta_i} P_i(\ell + \log V)\right) - 1. \tag{11.5}$$

`RealInverseResidual` evaluates this identity in the truncated jet algebra. A zero result below relative weight B establishes algebraic cancellation of all checked coefficients. It is not a claim that the exact residual is identically zero or that a numerical enclosure was computed.

There is also a useful sign-and-slope check. If the first missing inverse layer is $t^{1+\lambda}Q_\lambda(\ell)$, then the leading normalized residual of the retained approximation is

$$\mathcal{R}(t) = -p\, t^\lambda Q_\lambda(\ell) + o(t^\lambda). \tag{11.6}$$

This tests the derivative normalization, the sign of reversion, and the frontier polynomial at once. It is among the independent exact checks.

11.3 Using a finite forward asymptotic jet

The implementation treats its finite input as an *exact function.* Suppose instead that a true forward function is

$$F(x) = ax^p(1 + H_0(x) + R(x)),$$

where H_0 is the finite model supplied to the package and, for some $\rho > 0$ and nonnegative integer s,

$$R(x) = \mathcal{O}(x^\rho(1 + |\log x|)^s), \qquad xR'(x) = \mathcal{O}(x^\rho(1 + |\log x|)^s). \tag{11.7}$$

These assumptions imply the same positive slope asymptotic and the existence of the inverse germ. If g_0 is the inverse of the finite model, evaluating $F(g_0) - y$ and using Equation (11.2) gives

$$F^{-1}(y) - g_0(y) = \mathcal{O}(t^{1+\rho}(1 + |\log t|)^s). \tag{11.8}$$

More generally their qth powers differ by $\mathcal{O}(t^{q+\rho}(1 + |\log t|)^s)$.

Therefore every inverse coefficient at relative weight strictly below ρ is determined by the supplied forward jet. At the boundary, unknown forward information can alter the omitted polynomial. Simply applying `Normal` to a `SeriesData` input and discarding its remainder does not justify arbitrarily high inverse orders. The package rejects an unparsed `SeriesData` or explicit error term; management of a forward remainder belongs to the caller unless the exact finite model is genuinely the intended function.

11.4 Newton refinement

Once a positive approximation G is available, a Newton correction is

$$G_{\text{new}} = G - \frac{f(G) - y}{f'(G)}.$$

For the finite model, on a sufficiently small comparability interval, $f''/f' = \mathcal{O}(1/t)$. If $G - g = \mathcal{O}(t^{1+\beta}(1 + |\log t|)^d)$, Taylor's theorem gives

$$G_{\text{new}} - g = \mathcal{O}(t^{1+2\beta}(1 + |\log t|)^{2d}).$$

This is an optional numerical refinement or a possible future formal backend. It is not a third implemented package method. A numerical implementation should keep a positive bracket when needed, rather than assume that an unconstrained root finder will preserve the branch.

12 The Wolfram Language package

12.1 Loading and immediate use

The package is self-contained and has no third-party Wolfram dependencies. After extracting the archive, load its kernel file:

```
Get["path/to/RealInverseAsymptotics/Kernel/RealInverseAsymptotics.wl"];
Clear[x, y];

r1 = RealInverseSeries[x + x^2 (1 + Log[x]), {x, 0}, y, 6];
r1["Expression"]
r1["Remainder"]

r2 = RealInverseSeries[x + x^Sqrt[2], {x, 0}, y,
                       4 Sqrt[2] - 3];
r2["Expression"]
```

For `r1`, target powers strictly below 6 are retained, so the expression is Equation (1.3) without its remainder. Its remainder metadata has target power 6 and logarithmic degree 5. For `r2`, the cutoff is exactly the next power after the four requested terms; that next term is not retained.

The direct model interface avoids relying on symbolic parsing:

```
Clear[ell, y];
r = InversePowerLog[2, 3, {{1/2, 1 + ell}}, ell, y, 5/6];
r["Expression"]
```

This produces Equation (10.2) through its first three terms. The log placeholder `ell` is independent of the target `y`. Variables used as source, target, and placeholder should have no assigned values.

12.2 Public functions

Function	Contract
`PowerLogNormalForm [f,x]`	Parse an exact finite power–logarithmic expression into coefficient a, leading power p, and correction pairs $\{\delta_i, P_i\}$.
`RealInverseSeries [f,{x,0},y,b]`	Return a branch-aware inverse result association, keeping target powers strictly below b.
`InversePowerLog [a,p,corr,L,y,b]`	Use explicitly supplied normalized model data; `corr` is a list of pairs.
`RealInverseResidual [result,B]`	Compute the finite normalized residual jet at relative t-powers strictly below B. Requires the ordinary inverse, not another observable power.
`PurePowerInverseCoefficient [alpha,n]`	Return Equation (9.3) for exact real algebraic $\alpha > 1$ and integer $n \geq 0$.
`PowerLogRemainder [t,r,d]`	Inert ordinary big-O metadata, $\mathcal{O}(t^r(1 + \lvert\log t\rvert)^d)$.

12.3 Result fields and their coordinate systems

The following fields are important when using a result programmatically.

Field	Meaning
`"Expression"`	The explicit finite expression in y, with logarithms evaluated at $\log(y/a)/p$.
`"Terms"`	Pairs $\{r, Q(\ell)\}$ meaning $t^r Q(\ell)$, not $y^r Q(\log y)$. The basis is recorded in `"TermBasis"`.
`"Coefficient"`, `"LeadingPower"`	The forward leading data a and p.
`"Corrections"`, `"LogVariable"`	The normalized input and its polynomial placeholder.
`"Cutoff"`	The strict target-variable power cutoff b.
`"ObservablePower"`	The q in $g(y)^q$; default 1.
`"FrontierWeight"`	The relative normalized weight λ.
`"FrontierPolynomial"`	The combined Q_λ; it may be zero.
`"FrontierExpression"`	The first possible omitted layer expressed in y, supplied for nonexact results.
`"RemainderPower"`	The target-variable power $(q + \lambda)/p$.
`"RemainderLogDegree"`	The degree of Q_λ, or zero for a cancelled frontier.
`"Remainder"`	The inert descriptor in the normalized t coordinate.

Field	Meaning
`"IsExact"`	True for the exact monomial inverse or the constant observable $q = 0$; otherwise exactness is not claimed.
`"Branch"`, `"Assumptions"`	The positive germ and the parameter/target conditions under which the result is intended.
`"EnumeratedMultiIndices"`	Number of simplex points enumerated, including the zero multi-index.

The distinction among b, B, and λ is intentional. For $a = p = q = 1$, the simple relation $B = b - 1$ can hide a coordinate mistake that becomes visible only for a nonlinear leading power. The residual function uses a relative normalized cutoff; the inverse function uses an absolute target-power cutoff.

12.4 Options and symbolic parameters

The inversion functions accept `Assumptions`, `"ObservablePower"`, `"Method"`, `"MaxMultiIndices"`, and `"MaxTotalDegree"`. Defaults are `True`, 1, `"Lagrange"`, 100000, and 1000, respectively. The second implemented method is `"FixedPoint"`.

```
Clear[a, b, x, y];
r = RealInverseSeries[a x + b x^2, {x, 0}, y, 3,
      Assumptions -> a > 0 && Element[b, Reals]];
FullSimplify[r["Expression"], a > 0 && y > 0]
(* y/a - b y^2/a^3 *)

rSquared = RealInverseSeries[x + x^2, {x, 0}, y, 6,
                             "ObservablePower" -> 2];
```

These are fixed-parameter asymptotic statements. They do not assert uniformity as, for example, a positive leading coefficient tends to zero or an increment δ_i tends to zero. Such limits change the normalization or the number of layers needed at a fixed cutoff.

To cross-check methods and the residual:

```
rA = RealInverseSeries[x + x^2 (1 + Log[x]), {x, 0}, y, 5];
rB = RealInverseSeries[x + x^2 (1 + Log[x]), {x, 0}, y, 5,
                       "Method" -> "FixedPoint"];
FullSimplify[rA["Expression"] - rB["Expression"], y > 0]
(* 0 *)
RealInverseResidual[rA, 4]["AllZero"]
(* True: relative weights below 4 vanish *)
```

The comments above specify the mathematical expected outputs and are covered by the supplied test suite; they are not presented as a transcript from a native kernel run in this environment.

12.5 Accepted syntax and explicit failure cases

The expression parser accepts finite sums of products of constants, powers x^r, and nonnegative integer powers of $\log x$. It expands sums and products and then combines equal powers. A nonconstant polynomial in $\log x$ is allowed in correction sectors but not as the leading coefficient. The source variable and target variable must be distinct.

Unsupported inputs return a structured `Failure` through the supported call signatures. Examples include a zero function, a nonpositive leading power, an unproved positive leading coefficient, complex coefficients, floating-point coefficients or exponents, transcendental exponents such as π, nonpositive correction increments, negative logarithmic powers, nested logarithms, and unknown method names.

A cutoff at or below the leading target power is rejected. Resource limits stop the computation with a failure rather than return an unannounced partial expansion.

For instance, $x + x^{\pi}$ is mathematically covered by the theorem but not by the current exact-algebraic exponent backend. This is a deliberate implementation boundary, not a claim that its inverse cannot be expanded: Equation (9.3) applies directly with $\alpha = \pi$. Extending the backend requires a trustworthy comparison and equality oracle for the chosen larger class of constants.

13 Implementation structure, invariants, and cost

13.1 The construction pipeline

The parser first produces a finite list of exponent–polynomial pairs. It validates exact real algebraic exponents, sorts them by exact order, combines collisions, and identifies the least exponent. That least coefficient must be a provably positive constant. Dividing by it yields the normalized correction polynomials.

The inversion engine then computes $B = pb - q$ and enumerates the bounded simplex Equation (7.5) recursively. It finds the least weight at least B, keeps every multi-index at or below that frontier, computes Equation (4.3), and merges equal weights. The fixed-point backend uses the same cutoff and model but constructs the coefficients using only finite jet multiplication, logarithm, binomial power, and composition.

The final stage simplifies coefficient polynomials under the supplied assumptions, translates retained terms into y, and attaches the first-omitted-layer metadata. Exact monomial inputs and $q = 0$ return early. The package never calls `Solve`, `InverseFunction`, `InverseSeries`, or `PowerExpand` to choose or construct the inverse branch.

13.2 Core invariants

Every internal jet is a finite list of real algebraic weights with polynomial amplitudes. Equal weights are merged. Before a logarithm or binomial expansion, the nonconstant part has strictly positive weight. All arithmetic is truncated only after the relevant weight has been computed exactly. A source-level exponent must not survive validation as an unresolved symbolic inequality.

`RootReduce` supplies canonical algebraic reduction for exponent differences; its documented purpose and canonical behavior are described in [8]. The comparator tests the sign of the reduced exact difference. It never substitutes machine-precision approximations as a tie-breaker. Polynomial coefficients are simplified separately, so exponent arithmetic and coefficient algebra are not conflated.

Within the positive-power filtration, removing terms above a cutoff is safe for subsequent multiplication and normalized composition: those operations cannot decrease relative weight. Derivatives used in the explicit coefficient theorem are applied inside the closed formula before the resulting target power is assigned. Blindly differentiating a truncated absolute-power jet and retaining its old cutoff would not be safe; the package does not do that.

13.3 Size bounds and practical limits

If K is the number of enumerated multi-indices and $U = n_* \delta_*$, a simple bound is

$$K \le \prod_{i=1}^{m} \left(1 + \left\lfloor \frac{U}{\delta_i} \right\rfloor \right).$$

The simplex constraint is usually much smaller than this box bound. Nevertheless, many sectors or a very small δ_* can make a modest power cutoff expensive. Rational dependencies may cause many

multi-indices to merge into comparatively few final weights; the multi-index engine still has to account for their contributions.

For a multi-index of degree n, the amplitude degree is at most $\sum_i k_i d_i$. After forming its polynomial product, the $n-1$ first-order operators can be applied by linear passes over coefficient vectors as in Equation (4.6). The reference implementation favors transparent polynomial expressions over a specialized array backend. Its jet multiplication uses direct convolution and repeated sorting/merging; it is not an asymptotically optimized Newton-series library.

The explicit method is normally the natural first choice. The fixed-point method is valuable as an independent implementation and for studying composition, but repeated polynomial convolutions can cost more. For large problems, useful next steps are cached polynomial powers, coefficient-vector arithmetic, a support-indexed sparse map, an integer-lattice representation when a rational basis is available, and a Newton backend. None of these optimizations changes the coefficient or remainder theorem.

14 Validation and reproduction

14.1 Executed checks

The independent program `validation/validate.py` implements exact weight comparison for its fixtures, the differential-operator formula, and a separately coded truncated composition algebra. It uses SymPy 1.14.0 and mpmath 1.3.0 in the recorded run. Its 104 exact assertions include the displayed logarithmic coefficients, the irrational-power coefficients, agreement with the finite product formula, resonant collisions, nonlinear leading powers, two distinct irrational increments with mixed products, an irrational leading power, fractional and negative observables, normalized composition residuals, and the frontier sign-and-slope identity. Randomized fixtures use the fixed seed 236367 and are stored explicitly in `results.json`.

This program is independent of the Wolfram source. It is strong evidence for the coefficient formulas and finite-jet algorithms and it generated numerical examples reproducibly. It does *not* execute Wolfram syntax, test Wolfram evaluation rules, or validate the public parser. Those are separate obligations of the MUnit suite.

For the numerical comparisons, the inverse was approximated by 180-decimal-digit bisection inside a positive bracket. The program records the actual approximation error and its ratio to the first omitted term. The following table uses the logarithmic expansion through y^4, the irrational expansion through the $n=4$ correction, and Equation (10.2) through t^2. Values are rounded for display.

Family	y	$\lvert g-G\rvert$	$(g-G)/\text{frontier}$
Logarithmic	10^{-2}	1.48666×10^{-7}	1.0950093169
Logarithmic	10^{-4}	5.09642×10^{-16}	1.0022640153
Logarithmic	10^{-8}	1.16393×10^{-34}	1.0000005026
Logarithmic	10^{-12}	6.58956×10^{-54}	1.0000000001
Irrational power	10^{-2}	4.15553×10^{-6}	0.7805227225
Irrational power	10^{-4}	3.68347×10^{-12}	0.9597364481
Irrational power	10^{-8}	1.99266×10^{-24}	0.9990753832
Irrational power	10^{-12}	1.03646×10^{-36}	0.9999796056
Nonlinear core	10^{-12}	9.49699×10^{-9}	1.0887583784
Nonlinear core	10^{-24}	8.47300×10^{-18}	1.0017628576
Nonlinear core	10^{-48}	7.41352×10^{-37}	1.0000003656

Table 3: Executed 180-digit numerical comparisons. The finite approximations and first omitted terms are generated from exact coefficients.

The ratios approach 1, as predicted by Equation (7.3) when the frontier polynomial is nonzero. At the less asymptotic test points they need not be especially close to 1: higher layers can still matter. The exact proofs, not these samples, establish branch existence, convergence, and the stated ordinary big-O estimates.

14.2 Native Wolfram execution status

The remote Wolfram evaluator was attempted and returned `invalid_mcp_response`, with an HTTP 404 response from its service endpoint. A local Wolfram kernel was not available. Installing an alternative interpreter also failed because network name resolution was unavailable. Accordingly, no result in this deliverable is described as a successful native Wolfram execution.

The Wolfram source received a string- and nested-comment-aware delimiter scan and manual source review. These can detect some transcription mistakes but are not substitutes for a Wolfram parser or evaluator. The 95 MUnit tests in `Tests/RealInverseAsymptotics.wlt` are supplied for native execution. They include public API expectations, structured failure cases, resource limits, the two requested examples, and cross-backend/residual/observable checks for the deterministic fixtures. Their recorded status is *not run*.

This distinction matters when assessing readiness. The mathematics has self-contained proofs, and the underlying algorithms have executed independent checks. The delivered Wolfram code remains a reference implementation whose native integration tests must still be run in a working kernel; it is not represented as production-certified software.

14.3 Commands to reproduce the checks and the PDF

From the extracted project directory:

```
python validation/validate.py
wolframscript -file Tests/RunTests.wls
```

The Python command rewrites `validation/results.json` with the full check list and numerical results. In a Wolfram notebook, the equivalent native command is

```
TestReport["path/to/RealInverseAsymptotics/Tests/RealInverseAsymptotics.wlt"]
```

The test file loads the package relative to its own location. The runner prints succeeded and failed test counts and returns a nonzero exit code unless the failed-test count is zero.

The article uses a standard pdfLaTeX workflow, with Libertinus when available and Latin Modern as a fallback:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error real-inverse-asymptotics.tex
pdflatex -interaction=nonstopmode -halt-on-error real-inverse-asymptotics.tex
```

The source contains its bibliography directly; no external bibliography database or proprietary font file is required. The numerical table is a small included TeX source derived from the recorded results. Font files are not included in the archive.

15 Extensions and boundaries of the present engine

15.1 Other endpoints and asymptotic coordinates

An inverse near a finite point (x_0, y_0) can often be reduced to this problem by writing $x = x_0 + u$ and $y = y_0 + v$ and choosing a positive side for u and v. The resulting leading term must fit au^p with

$a, p > 0$. This is a caller-supplied coordinate change; the parser does not search for centers or branches.

For an inverse at infinity, reciprocal variables can sometimes produce an equivalent small-argument problem. One must transform the entire function and its domain, not simply reuse an expansion with y replaced by $1/y$. Leading logarithmic or exponential cores commonly require a different scale after such a transformation.

A left-hand real branch can also be reduced by $x = -u$ only when the original powers and logarithms have been assigned consistent real meanings on that side. For irrational powers, x^α on negative x is not automatically the same real-valued function as a signed power. The package makes no such convention implicitly.

15.2 Leading logarithms require a different core

Consider

$$f(x) = x \log(1/x), \qquad x \to 0^+.$$

Here the logarithm belongs to the leading scale, not a positive-power correction. With $u = -\log x$ the equation is $y = u\mathrm{e}^{-u}$, so the branch approaching zero is

$$x = -\frac{y}{W_{-1}(-y)}. \tag{15.1}$$

The branch W_0 would instead give x near 1. This is a particularly clear example of why the branch must be fixed by its asymptotic normalization rather than by the name of an inverse-function object.

For $ax^p(-\log x)^r$ with $r > 0$, the exact core may similarly be written

$$u = -\frac{r}{p} W_{-1}\left(-\frac{p}{r}(y/a)^{1/r}\right), \qquad x = \mathrm{e}^{-u}.$$

Subleading corrections can then be transported through this exact core by phase-coordinate or scaled Lagrange inversion, as developed in the supplied companion material [3]. The current package does not implement this Lambert-core backend. It explicitly rejects a logarithmic leading coefficient rather than return a power-core answer outside its hypotheses.

15.3 Infinite input expansions and flat corrections

The finite analytic lifting is not, by itself, a convergence proof for an arbitrary infinite forward expansion. For an infinite support, one needs additional local-finiteness and summability conditions, or a finite-jet statement with controlled remainders such as Equation (11.7). A list of plausible formal coefficients is not a substitute for those conditions.

Flat corrections also contain information invisible to every finite power–logarithmic jet. For example,

$$F(x) = ax^p(1 + \mathrm{e}^{-1/x})$$

has the same complete ordinary power–logarithmic asymptotic expansion as ax^p, but its inverse satisfies

$$F^{-1}(y) = t - \frac{t}{p}\mathrm{e}^{-1/t}(1 + o(1)), \qquad t = (y/a)^{1/p}.$$

The coefficient follows from the leading displacement $-\{F(t) - at^p\}/(apt^{p-1})$ and the smallness of the differentiated flat term. The two inverses are not equal merely because all of their power–logarithmic coefficients agree. Computing such a correction requires keeping its exact sector in the input and using a larger algebra; the present parser intentionally rejects it.

15.4 Toward a broader engine

The existing implementation already separates three concerns that should remain separate in an extension: a branch/coordinate normalizer, an ordered coefficient algebra, and a remainder or residual contract. Supporting negative powers of logarithms would enlarge the coefficient algebra from polynomials to a suitable asymptotic field. Supporting leading logarithms would replace the power core by an exact Lambert coordinate. Supporting exponential sectors would change the support ordering and differentiation rules. Supporting symbolic exponents would require conditional comparisons and possibly several parameter chambers.

None of those changes should be hidden inside a generic simplification call. A useful generalized solver should return the selected branch, the scale in which it expanded, and the assumptions under which its ordering decisions hold. The finite engine here provides a fully specified base case for that architecture, rather than a claim that the remaining cases are automatic.

16 Relation to the supplied research materials

The supplied `series-and-transseries` directory contains a much broader program concerning compositional inversion and transseries. For this report, the most directly relevant inspected source was the companion volume *Combinatorial Transseries and Their Inverses* [3]. In particular, its sections labeled `ct:sec:phase` and `ct:sec:scaled` distinguish an exactly invertible core from a perturbation and derive finite Lagrange formulas. Its discussion of marker convergence correctly emphasizes that a local identity near zero markers does not alone justify setting every marker to 1.

The snapshot inspected for those sections had Git blob identifier

`05c56ce5ee60788a67f365110b62810550cba817.`

The reference is to selected sections, not an assertion that the entire large directory or all of its research-frontier claims were audited.

The construction in this article specializes that strategy to a positive power core and polynomial logarithmic amplitudes, eliminates the derivatives in the original variable through the operators $q + \Delta + pj + \partial_\ell$, and supplies a finite analytic lifting and an exact cutoff algorithm for this class. The broader transseries literature [4, 5] motivates the scale-based viewpoint; the elementary local-finiteness and analytic arguments here make the stated results independent of unresolved questions about more general supports.

Classical Lagrange inversion is not claimed as new. The value of the present development is the concrete specialization, the branch and remainder contracts, the two independent finite algorithms, and a reproducible implementation package for the exact examples in the question.

A Compact reference algorithms

The following pseudocode summarizes the explicit backend. Its input amplitudes have already been combined at equal increments, zero amplitudes removed, and all branch and exactness checks performed.

```
input: a > 0, p > 0, corrections (delta_i, P_i(L)), observable q
       strict target-power cutoff b > q/p
B       := p*b - q
delta0  := minimum delta_i
upper   := ceiling(B/delta0)*delta0
indices := all k in N^m with sum(k_i*delta_i) <= upper
lambda  := minimum weight among indices with weight >= B
jet     := empty map from exact weights to polynomials
```

```
for each k with weight Delta <= lambda:
    n := sum(k_i)
    if n == 0:
        jet[0] += 1
    else:
        P := product(P_i(L)^k_i)
        for j = 1,...,n-1:
            P := (q + Delta + p*j)*P + derivative(P,L)
        jet[Delta] += (-1)^n*q*P/(p^n*product(k_i!))

combine and simplify each coefficient polynomial
retain Delta < B; substitute t=(y/a)^(1/p), L=Log[y/a]/p
return finite expression and the full polynomial jet[lambda]
```

A sparse normalized jet is an associative map from exact relative powers to polynomials. Its composition backend is summarized below. Every operation is truncated to the same relative bound.

```
V := 1
repeat floor(bound/delta0)+1 times, or until stabilization:
    U := V - 1
    logV := sum_{n>=1} (-1)^(n+1)*U^n/n
    H := sum_i t^delta_i * (1+U)^delta_i * P_i(L+logV)
    V := (1+H)^(-1/p)
return V^q
```

All infinite sums displayed in this pseudocode terminate after truncation because their arguments have positive relative order. The actual source also performs structured validation, resource checks, exact exponent comparison, and final coefficient simplification.

B A checklist for adding a new test family

A new family should specify the positive domain and the exact leading core first. It should then compare the explicit and fixed-point methods, compose the retained inverse into the exact forward model, and check the first residual coefficient against Equation (11.6). At least one test should place the cutoff exactly on a generated weight and one should place it between two weights. A family with rational dependencies should include a deliberate exponent collision; one with logarithms should check both the polynomial coefficient and its logarithmic remainder degree.

Numerical checks should use higher precision than the expected inverse error, a bracket selecting the intended real branch, and several geometrically separated target values. An observed small residual must be interpreted through the appropriate slope. A zero formal frontier must not be mislabeled a nonzero asymptotic leading error. These checks exercise the mathematical contract rather than merely compare printed expressions.

C Archive contents

Path	Purpose
`article/real-inverse-asymptotics.tex`	Main article source.
`article/real-inverse-asymptotics.pdf`	Rendered article.
`article/numeric-table.tex`	Reproducible numerical table.
`Kernel/RealInverseAsymptotics.wl`	Wolfram Language reference package.
`Kernel/init.m`	Relative package loader.

Path	Purpose
`Examples/Examples.wl`	Runnable example expressions.
`Tests/RealInverseAsymptotics.wlt`	Native MUnit test specifications.
`Tests/RunTests.wls`	Native test runner.
`validation/validate.py`	Independent exact and numerical checks.
`validation/results.json`	Executed check list and recorded data.
`validation/REPORT.md`	Scope and status of validation.
`validation/static-check.py`	Delimiter scan, explicitly not an interpreter.
`README.md`, `LICENSE`	Usage, limitations, and license for original material.

References

[1] Vladimir Reshetnikov, *How to get an asymptotic of the real-valued branch of the inverse function?*, Mathematica Stack Exchange, question 236367 (2020), user-supplied archive and online question, accessed 7 September 2026. https://mathematica.stackexchange.com/questions/236367/how-to-get-an-asymptotic-of-the-real-valued-branch-of-the-inverse-function

[2] NIST Digital Library of Mathematical Functions, §1.10(vii), *Inverse Functions*, including the Lagrange inversion formula. Accessed 7 September 2026. https://dlmf.nist.gov/1.10.vii

[3] Vladimir Reshetnikov, *Combinatorial Transseries and Their Inverses*, consolidated companion volume in the ProveIt repository, September 2026; selected sections on phase-coordinate inversion, scaled local reversion, marker convergence, and coefficient calculus. Source snapshot identified in Section 16. https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/docs/semi-formalized-research-frontiers/drafts/series-and-transseries/Combinatorial_Transseries_Inverses/Combinatorial_Transseries_Inverses.tex

[4] G. A. Edgar, *Transseries for Beginners*, Real Analysis Exchange **35** (2010), 253–310; arXiv:0801.4877. https://arxiv.org/abs/0801.4877

[5] G. A. Edgar, *Transseries: Ratios, Grids, and Witnesses*, arXiv:0909.2430 (2009). https://arxiv.org/abs/0909.2430

[6] Wolfram Research, *InverseSeries*, Wolfram Language & System Documentation Center. Accessed 7 September 2026. https://reference.wolfram.com/language/ref/InverseSeries.html

[7] Wolfram Research, *AsymptoticSolve*, Wolfram Language & System Documentation Center. Accessed 7 September 2026. https://reference.wolfram.com/language/ref/AsymptoticSolve.html

[8] Wolfram Research, *RootReduce*, Wolfram Language & System Documentation Center. Accessed 7 September 2026. https://reference.wolfram.com/language/ref/RootReduce.html